EXAMEN SISTEME DE OPERARE

Grile:

- La terminare, un program returnează => un număr întreg
- Planificarea la execuție a programelor este => o funcționalitate oferită de SO
- Un proces aflat în execuție care apelează read() pe capătul de citire dintr-un pipe în care nu sunt date va fi trecut în starea => blocat
- Ieșirea standard se poate redirecta către un fișier pentru care există deja fd astfel => dup2(fd, 1)
- O variabilă de mediu bash => e de tipul șir de caractere
- Comanda wc -l numără => liniile primite la intrarea standard
- Nr. de blocuri de date ocupate pe un fișier pe disc se poate afla => citind o informație din nodul index și aplicând o formulă care depinde și de dimensiunea blocului
- /dev/null => fișier special spre care orice date sunt scrise se pierd
- bash => un shell/un interpretor de comenzi
- Un blu-ray care are acces la servicii de streaming are SO? => da, pentru că e un sistem de calcul cu procesor și dispozitive periferice
- fork() => un apel care creează un proces (nou)
- Java Virtual Machine e o mașină virtuală => tip proces
- Informațiile despre drepturile de acces se găsesc => în nodul index asociat acestuia
- Pentru a modifica drepturile de acces => chmod

- kill() => trimite un semnal
- Legăturile fixe sunt => intrări la director diferite care referă același nod index
- Un șir de dim. variabilă < 100 transmisă prin pipe => write(pfd[1], s, strlen(s))
- Informațiile despre nr. de legături spre un fișier se găsesc => în nodul index
- Pentru ca un proces parinte sa fie informat de terminarea procesului-fiu, semnalul SIGCHLD trebuie sa fie luat in considerare in mod special => tratat in procesul parinte
- ./program ”1 2” ”3 1” ”2 3” ”1” 2 3, argv[2] => 3 1
- Codul pentru o funcție bibliotecă se află => fișier binar
- Intrarea de dubla indirectare aflata in tabelul din nodul index refera => un bloc continand adrese de bloc
- Un pipe trebuie creat într-un program => înainte de a crea toate procesele care au nevoie de el
- Creare folder în UNIX => mkdir
- Nr. de legături al unui fișier se găsește în => inode
- Router poate avea SO? => da, pentru că e un sistem de calcul cu procesor și dispozitive periferice
- echo “cat f | wc-l” > g => scrie intr-un fisier cat f\ | wc-l
- echo ‘cat f | wc-l’ > g => scrie intr-un fisier numarul de linii al unui alt fisier
- Televizor non-smart care poate rula filme de pe USB are SO? => da, pentru că e un sistem de calcul cu dispozitive periferice
- Daca un semnal e trimis unui proces, atunci => daca procesul trateaza semnalul si acest lucru e posibil, se apeleaza o functie din acel proces

- Diferenta dintre execlp() si execl() este => execlp tine cont de date memorate intr-o anumita variabila de mediu
- Un proces este => un program sau o parte de program aflata sub supervizarea SO
- Ce combinatie de comenzi va schimba directorul curent? => cd
- x = 10, xx afiseaza => x10$
- signal() => specifica pentru procesul curent modul de tratare a unui semnal
- for(int i = 1; i < n; i++)
  fork(); cate procese sunt create => 2^{n-1}
- Symlink path, bytesize => lstat
- var = 3;

  echo $var && {var = “abc”; echo “$var”; }

  echo $var

  va afisa => 3, abc, abc

- Un proces citeste din mai multe pipe-uri si trebuie sa trateze intotdeauna primele date disponibile din oricare dintre pipe-uri. Pt asta, folosim => select()
- Cand un proces A creaza un nou proces B folosind fork(), B este => proces copil
- var = 23
  echo ‘ `expr $var + 3` ’
  echo “ `expr $var +3` “
  afiseaza `expr $var + 3` , 26
  ‘ ‘ – vezi exact ce e intre ele
  “ “ – vezi ce e intre ele, dar variabilele isi iau valorile
  ` ` - se pun comenzi in interiorul ei

- ls => comanda externa
- grep -E “^.[st].+\.t[enpxt]*$” => pst.txt
- Un fisier poate fi facut disponibil pentru doua procese astfel => un proces creeaza fisierul si apoi il creeaza pe celelalt proces
- Daca un proces trimite intr-un pipe o structura de date => un alt proces poate citi doar jumatate din structura, in anumite conditii
- Grep => gaseste potriviri pentru un tipar
- Cand un proces A creaza un proces B cu fork(), dupa ce A se sfarseste, procesul B este => orfan
- Functie sistem UNIX pentru sters fisier => unlink
- creat() => creeaza un fisier
- Deschiderea unui fisier pentru citire => open(file, O_RDONLY);
- for(int i = 0; i <= 2; i++) => 3 valori
  fork(); 2^3-1(procesul parinte) 7 procese noi create
- sh script.sh 1 2 3 => argv[0] este script.sh
- execlp() => executa un program de pe disc in cadrul procesului curent
- O comanda externa este => un fisier executabil, aflat intr un director a carui cale se afla in variabila de mediu PATH in mod implicit
- Un fir de executie in UNIX este => o executie secventiala in cadrul unui proces
- Flag-urile SETUID si SETGID se refera la => fisiere aflate in sistemul de fisiere
- Daca un proces trimite cu succes date unui alt program prin pipe, inseamna in mod necesar ca => procesele nu au inchis capetele utile ale pipe-ului la care au acces

- Asteptarea terminarii unui fir de executie de tip “detached” => nu se poate realiza

RASPUNS MULTIPLU:

- SO:
  - are acces la spatiul nucleu
  - ofera o interfata de programare pt aplicatii
- Stiva e privata fiecarui proces
- read() returneaza un nr mai mic decat al treilea parametru:
  - cand se citeste ultimul set de date dintr-un fisier
  - in caz de eroare
  - cand fisierul citit e mai mic in bytes decat nr de octeti ceruti
- Nucleul unui sistem de operare bazat pe arhitectura microkernel:
  - format din mai multe servere independente ce ofera servicii
  - ofera servicii de baza, cum ar fi gestiunea proceselor
  - e parte a unei arhitecturi flexibile, cu mentenanta usoara
- Un proces UNIX poate contine mai multe fire de executie
- Comanda chmod 512 x va avea urmatorul efect pentru fisierul x:
  - drept de read pentru proprietar
  - drept de executie pentru grup

- Un proces A creeaza un nou proces B, dupa care A apeleaza cu succes execlp pentru a porni un program P:
    - Programul nu va mai putea prelua explicit valoarea returnata la terminare de B
    - Procesul care ruleaza P va avea acelasi pid ca A
- alarm():
    - programeaza trimiterea unui semnal anume in viitor
    - reseteaza o alarma declansata
- la UNIX un fir de executie poate daca e programat in acest scop
    - termina procesul curent
    - apela functii
    - returna o valoare
    - memora o valoare
    - accesa si modifica valorile variabilelor globale ale procesului in care firul a fost creat
- Un pipe lucreaza:
    - pe principiul FIFO
    - un canal de date unidirectional
- SO UNIX:
    - management de memorie
    - interfata de programare
    - gestiunea procesului

- Exemple SO:
    - Windows
    - MS-DOS
    - Tizen
    - Linux
    - Firefox OS
    - Android
- In UNIX, programele scrise de utilizator:
    - Pot transmite date catre alte programe
    - Pot redirectiona singure iesirea standard
    - Pot fi formate din mai multe procese
- open():
    - apel sistem
    - o functie aflata in interfata de programare UNIX
- Apelul corect pentru a crea un fisier cu scop de a scrie in el:
    - creat(file, S_IRUSR | S_IWUSR)
    - open(file, O_WRONLY | O_CREAT | O_TRUNC, S_IRUSR | S_IWUSR)
- SO ofera un API
- Un proces poate obtine intotdeauna privilegii de root daca:
    - UID real al procesului este root
    - UID sau salvat este root
- Este recomandata folosirea bufferelor de dimensiunea unui bloc pentru optimizare atunci cand:
    - se citeste sau scrie intr un fisier

- O rutina de tratare poate:
    - Trimite un mesaj
    - Trata mai multe semnale diferite, facand uz de argumentul primit
    - Termina procesul curent
- Un proces mosteneste de la procesul pe care l-a creat:
    - Toate variabilele globale
    - Fisierele deschide inainte de creare
    - Redirectarile de descriptori efectuate inainte de creare
- Pentru a afla spre ce cale indica o legatura simbolica si dimensiunea legaturii de pe disc e nevoie de functiile:
    - Readlink
    - Lstat
- Daca un proces trimite intr-un pipe o structura de date:
    - un alt proces poate citi doar jumtate din structura, in anumite conditii
    - poate exista un proces care poate citi toata structura
- Un fisier poate fi facut disponibil pentru doua procese astfel:
    - Procesul care deschide fisierul trimite descriptorul de fisier obtinut la open() celuilalt
    - Un proces creeaza fisierul apoi il creeaza pe celelalt proces
    - Ambele procese cunosc descriptorul de tip INT al fisierului